Video 02 — How hardware enables AI

Read only the narration paragraphs. Section headings, numbers and timestamps are not spoken. Timings refer to the original edit; speak naturally.

01. Pip’s plan is waiting | 00:00–00:29

Pip has a plan. The wheels still wait.

A sensor stream reaches our campus rover faster than its computer can process it.

A clever decision still needs memory, processing, connections, and enough power to become an action.

Have you ever wondered what gives an AI decision a physical body?

Chapter three, Hardware, will help us follow the whole path and find the actual bottleneck.

02. Today’s route | 00:29–01:04

You will trace the sensing, memory, processing, and actuation path of a small robot.

You will explain why memory movement can limit a fast processor and what a pipeline actually improves.

You will also choose hardware by its job, rather than assuming every AI system needs the same chip.

Welcome to the tutorial on how hardware enables AI by Blue Impulse.

Follow each part of the diagram, then predict what happens when that part becomes the bottleneck.

03. Follow the physical path | 01:04–01:52

The subtopic Hardware gives software a body groups hardware by input, processing, output, and storage.

For our robot, the sensor supplies observations. Memory holds inputs, model parameters, and working results.

The processor evaluates the program or model, and an output communicates the resulting decision.

If the robot needs to move, an actuator must turn that decision into physical motion.

Notice that one device can have multiple roles. A touchscreen both displays output and senses input.

The diagram is a chain of responsibilities, not a claim that every box must be a separate physical chip.

04. Use the kitchen analogy | 01:52–02:35

Use a kitchen to remember the memory hierarchy.

Registers are like the tiny workspace immediately in your hands. Cache keeps useful ingredients close to the cook.

Main memory is the larger active workbench, while persistent storage is more like the pantry.

The analogy is about access and capacity, not a literal map of computer components.

A huge pantry does not help if ingredients arrive too slowly. Similarly, a processor can wait for data.

Capacity asks how much fits. Memory bandwidth asks how quickly data can move. These are different questions.

05. The processor may be waiting | 02:35–03:47

Model weights, inputs, and intermediate results must fit somewhere. A processor cannot use data that has not arrived.

When memory bandwidth is insufficient, a fast processor may spend time waiting instead of doing useful arithmetic.

Adding storage capacity alone does not automatically remove a memory-transfer bottleneck.

The table also distinguishes cache, main RAM, and secondary storage. They are not interchangeable names for one resource.

Both static RAM and dynamic RAM are volatile, although their circuits and refresh requirements differ.

For this tutorial, focus on the system question: is the limiting factor how much fits, or how quickly it arrives?

Imagine the processor as a cook waiting for ingredients. Hiring a faster cook does not shorten the time a missing ingredient spends in transit.

Likewise, a faster arithmetic unit can still spend time waiting for memory. Measure that waiting before choosing the upgrade.

06. Match the processor to the work | 03:47–04:39

Now compare the processor roles in the subtopic Hardware for AI.

A CPU handles general control, branches, and diverse tasks. A GPU can accelerate suitable parallel arithmetic workloads.

Specialised accelerators can efficiently perform supported tensor operations, but their benefits depend on the actual workload.

Simple tabular reinforcement learning can run on an ordinary CPU. AI does not always require a GPU.

Training changes model parameters. Inference uses an existing model to produce an output.

A model can be trained on a large server and deployed on a smaller device if the deployment requirements fit.

07. Overlap work, increase throughput | 04:39–05:45

Use laundry to understand pipelining. One load can dry while a later load is washing.

In our simplified processor, the stages are fetch, decode, execute, and write.

Instruction one enters fetch in cycle one. In cycle two, it decodes while instruction two enters fetch.

Keep stepping forward. The instructions occupy different stages, so several instructions make progress during the same cycle.

After the pipeline fills, this ideal example completes one instruction per cycle, assuming no stalls.

It does not make an individual four-stage instruction take just one cycle from start to finish.

Real pipelines can pause for dependencies, uncertain branches, or shared resources. These hazards limit the ideal improvement.

Use the included Python table to compare completion cycles rather than relying only on the moving picture.

08. A signal is not motor power | 05:45–06:50

Trace the environmental control loop: measurement becomes a signal, processing chooses an output, and an actuator changes the environment.

An analogue sensor may need an analogue-to-digital converter. Many sensors already provide digital measurements.

The processor evaluates a decision, then a driver or power stage supplies what the actuator needs.

Some actuators use a digital interface or pulse-width modulation. They do not all require a digital-to-analogue converter.

Pulse-width modulation is timed switching, not the same thing as converting a value into an analogue voltage.

A control signal can tell a motor what to do without being able to supply the current that moves it.

Pip's control signal communicates what the motor driver should do. The energy that turns a loaded wheel still comes through the power system.

09. Close the loop | 06:50–07:35

Imagine the robot detects an obstacle, selects a turn, and sends a command to its motor driver.

The driver powers the motor, the wheels move, and the sensor measures the changed situation.

That final measurement closes the feedback loop. The command alone does not prove the movement succeeded.

A wheel might slip, a sensor might be noisy, or the actuator might reach a mechanical limit.

Monitoring and control differ. Displaying a measurement monitors; changing the environment adds control.

This loop can use a fixed rule or a learned policy. The hardware path alone does not tell you which.

10. Will a faster GPU fix this? | 07:35–08:32

Pause and diagnose a system. Its processor repeatedly waits for data, and its motor driver cannot supply enough current.

Would replacing the processor with a faster GPU solve both problems? Think before the answer appears.

No. Memory movement and actuator power are separate limits that a new GPU does not automatically remove.

Measure the path, identify the limiting stage, and match the intervention to that evidence.

Chapter six, Strengths and Limitations, adds power, heat, memory, and physical constraints.

An advertised arithmetic rate describes one capability. It is not a complete measurement of the robot's useful performance.

11. Hardware enables every step | 08:32–09:32

Trace Pip's whole path: observations enter, memory holds information, processing evaluates it, and outputs communicate or act.

Memory capacity and bandwidth both matter. Pipeline throughput is different from the latency of one instruction.

Conversion, control signals, actuator power, and processor strengths all affect what the system can do.

Choose hardware by the workload, deadlines, power budget, and physical task.

To check your understanding, draw a robot's path from camera to wheel, and label one possible bottleneck at each stage.

Then explain why a more powerful chip does not, by itself, create a learning algorithm.

An upgrade should target the constrained part of the task. Check the input rate, memory traffic, compute load, and actuator response as a connected system.

12. Pip completes the shift | 09:32–10:00

Pip's small problem now has an explanation you can test.

Use Chapter 3, Hardware, and the named subtopics to revisit the reasoning.

The tutorial guide includes the full teaching steps and the checked practice answers. Run the supporting demonstration and change one input.

Blue Impulse. Follow the signal, predict the result, and check your explanation.